

VeloShop - internetowy sklep rowerowy

1. Zagadnienia biznesowe

1.1 Opis problemu

VeloShop to internetowa platforma sprzedaży rowerów i akcesoriów rowerowych, obsługująca pełen cykl życia zamówienia: od przeglądania katalogu, przez złożenie zamówienia i płatność, aż po ewentualną naprawę lub serwis roweru. System kierowany jest zarówno do klientów indywidualnych, jak i do personelu serwisu rowerowego.

1.2 Aktorzy systemu

- Klient (niezalogowany) - przegląda katalog rowerów, widzi ceny i dostępność
- Klient (zalogowany) - składa zamówienia, śledzi status, zgłasza naprawy
- Administrator / Pracownik - Zarządza katalogiem produktów, obsługuje zamówienia i serwis

1.3 Główne procesy biznesowe

- Zakup roweru
- Zarządzanie katalogiem
- Obsługa zamówień
- Serwis i naprawa
- Autentykacja

Etap 2 - Wymagania funkcjonalne

2.1 user-service

Odpowiada za rejestrację, logowanie i zarządzanie profilami użytkowników. Wydaje tokeny JWT podpisane wspólnym sekretem. Udostępnia też `authMiddleware.js` używany przez pozostałe serwisy do weryfikacji tożsamości. Obsługuje dwie role: `customer` i `admin`.

- `POST /auth/register` - rejestracja nowego użytkownika

```
{
  "firstName": "Jan",
  "lastName": "Kowalski",
  "email": "jan@example.com",
  "password": "haslo123"
}
```

- POST /auth/login - logowanie, zwraca token JWT

```
{
  "email": "jan@example.com",
  "password": "haslo123"
}
```

- GET /users/me - profil zalogowanego użytkownika (wymaga JWT)

2.2 product-service

Przechowuje katalog rowerów i akcesoriów. Obsługuje stany magazynowe i umożliwia filtrowanie po kategorii, marce i cenie. Udostępnia też endpoint wewnętrzny do rezerwacji towaru używany przez order-service i repair-service.

- GET /products - lista produktów, query params: ?
category=&brand=&minPrice=&maxPrice=&available=
- GET /products/:id - szczegóły produktu
- POST /products - dodanie produktu (wymaga JWT, rola admin)

```
{
  "name": "Trek Marlin 5",
  "brand": "Trek",
  "category": "MTB",
  "price": 2499,
  "stock": 10,
  "description": "...",
  "imageUrl": "..."
}
```

- DELETE /products/:id - usunięcie produktu (wymaga JWT, rola admin)
- POST /products/:id/reserve - rezerwacja sztuk towaru (*endpoint wewnętrzny, między serwisami*)

```
{ "quantity": 2 }
```

2.3 order-service

Obsługuje pełen cykl zamówienia: przyjęcie koszyka, weryfikacja dostępności w product-service, zainicjowanie płatności w payment-service, finalizacja. Administrator może ręcznie zmieniać status wysyłki.

- POST /orders - złożenie zamówienia (wymaga JWT)

```
{
  "items": [{ "productId": 3, "quantity": 1 },
  { "productId": 7, "quantity": 2 }],
  "deliveryAddress": "ul. Rowerowa 1, Warszawa",
}
```

```
"paymentMethod": "card"
}
```

- GET /orders - lista zamówień zalogowanego użytkownika (wymaga JWT)
- GET /orders/:id - szczegóły zamówienia (wymaga JWT)

2.4 payment-service

Mockowa bramka płatności. Symuluje przetwarzanie transakcji.

- POST /payments - zainicjowanie płatności

```
{ "orderId": 15, "amount": 2499.00, "paymentMethod": "card" }
```

2.5 repair-service

Obsługuje zlecenia serwisowe niezależnie od modułu sprzedaży. Klient zgłasza usterkę, opisując rower (może wskazać produkt z katalogu Product Service lub podać własny opis). Serwisant aktualizuje status i dodaje notatki. Serwis odpytuje Product Service po dostępne części zamienne.

- POST /repairs - zgłoszenie naprawy (wymaga JWT)

```
{
  "productId": 3,
  "bikeDescription": "Trek Marlin 5, 2022",
  "issueDescription": "Pęknięta rama, przeskakujące biegi"
}
```

- GET /repairs - lista zleceń; klient widzi swoje, admin/serwisant widzi wszystkie (wymaga JWT)
- PATCH /repairs/:id/status - zmiana statusu (wymaga JWT, rola admin)

```
{ "status": "in_progress" }
```

2.6 frontend-react

Interfejs obsługujący wszystkie widoki klienta i administratora.

- Strona główna z polecanymi produktami
- Katalog rowerów z filtrowaniem i wyszukiwaniem
- Karta produktu ze szczegółami i przyciskiem „dodaj do koszyka”
- Koszyk i proces zakupowy krok po kroku (koszyk > dane dostawy > płatność > potwierdzenie)
- Panel użytkownika: historia zamówień, profil, zgłoszenia serwisowe
- Panel administratora: zarządzanie produktami, zmiana statusów zamówień i napraw
- Formularz zgłoszenia naprawy i widok śledzenia statusu

3. Architektura aplikacji

System zbudowany jest w architekturze mikroservisów. Każdy serwis działa jako niezależna aplikacja Node.js/Express z własną bazą danych SQLite (przez Sequelize). Frontend React komunikuje się bezpośrednio z poszczególnymi serwisami przez REST API.

Tożsamość użytkownika przekazywana jest za pomocą tokenu JWT w nagłówku `Authorization: Bearer <token>`.

Diagram komunikacji między serwisami:

